

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquelos.

Para el desarrollo del Ciclo 4 se definieron los siguientes mini-ciclos:

Mini-ciclo 1:

Se convirtieron StackingItem, Cup y Lid en clases abstractas, y se crearon las subclases NormalCup, OpenerCup, HierarchicalCup, NormalLid, FearfulLid, CrazyLid y StickyLid. Justificación: era necesario establecer la jerarquía de herencia antes de implementar cualquier comportamiento nuevo, ya que las demás etapas dependen de esta estructura.

Mini-ciclo 2

Se modificó Tower para soportar los nuevos tipos con pushCup(type, i) y pushLid(type, i), incluyendo la lógica de OpenerCup, HierarchicalCup, FearfulLid, CrazyLid y StickyLid. Justificación: concentrar el comportamiento polimórfico en Tower permite que las subclases permanezcan simples y el sistema sea fácil de extender.

Mini-ciclo 3

Se escribieron TowerC4test, TowerCC4test y TowerAtest cubriendo todos los nuevos tipos y sus comportamientos. Justificación: las pruebas garantizan que los nuevos requisitos funcionales se cumplan y que no se hayan roto funcionalidades existentes.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Los tres mini-ciclos definidos fueron completados satisfactoriamente:

- Mini-ciclo 1 completado: la jerarquía de herencia está implementada y funcional.
- Mini-ciclo 2 completado: Tower maneja correctamente todos los tipos nuevos con sus comportamientos especiales.
- Mini-ciclo 3 completado: las pruebas unitarias, colectivas y de aceptación fueron escritas y compilaron correctamente.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Cortes-20 horas

4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?

El mayor logro del ciclo fue la implementación exitosa del polimorfismo a través de la jerarquía de herencia. Al convertir Cup y Lid en clases abstractas y crear subclases con comportamientos distintos, el sistema se volvió mucho más extensible sin necesidad de modificar las clases base.

Esto se evidencia en cómo Tower maneja los diferentes tipos: en lugar de tener condicionales complejos dispersos por todo el código, la creación de objetos está centralizada en createCup() y createLid(), y el comportamiento especial de cada tipo está claramente delimitado. Este diseño facilita agregar nuevos tipos en el futuro sin romper el código existente, aplicando el principio Open/Closed de la programación orientada a objetos.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue diseñar el comportamiento de HierarchicalCup correctamente. La taza debía desplazar elementos de menor tamaño al entrar y, si llegaba al fondo, volverse inamovible. El reto estuvo en determinar el índice correcto de inserción dentro de la lista de items y en cómo representar el estado "llegó al fondo" de forma limpia. La solución fue agregar el atributo reachedBottom en HierarchicalCup con sus métodos setReachedBottom() y hasReachedBottom(), y crear el método privado pushHierarchical() en Tower que calcula el índice de inserción recorriendo la lista y buscando el último elemento cuyo tamaño sea mayor o igual al de la taza que entra. Si el índice calculado es 0, significa que llegó al fondo y se marca como inamovible.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Lo que se hizo bien:

- Se siguió el flujo de trabajo correcto: primero el diseño en Astah y luego la implementación en código.
- Se aplicaron principios de POO como herencia, encapsulamiento y polimorfismo de forma explícita y coherente.
- Las pruebas cubren todos los tipos nuevos incluyendo casos exitosos y fallidos.
- El código está documentado con Javadoc en todos los métodos.

Compromisos de mejora:

- Mejorar la cobertura visual de las pruebas de aceptación con más pausas y mensajes descriptivos.
- Explorar el uso de interfaces además de herencia para mayor flexibilidad en ciclos futuros.

- Organizar el repositorio Git con commits más frecuentes y mensajes descriptivos por cada mini-ciclo.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica XP más útil fue el desarrollo guiado por pruebas (TDD - Test Driven Development). Escribir las pruebas en paralelo con la implementación permitió detectar errores en los comportamientos especiales de los nuevos tipos de forma temprana, especialmente en casos como FearfulLid (que no debe salir si está tapando su taza) y StickyLid (que solo puede quitarse con removeLid). Sin las pruebas, estos comportamientos habrían sido difíciles de verificar manualmente dado que dependen de interacciones específicas entre los objetos. Las pruebas también funcionaron como documentación ejecutable del comportamiento esperado del sistema.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Escuela Colombiana de Ingenieria. (2026). Enunciado Proyecto Inicial Ciclo No. 4 2026-1: Refactoring y Extension. Documento de curso DOPO-POOB.

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley. Referencia principal para el uso correcto de herencia, clases abstractas e interfaces en Java.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. Consultado para el patrón Factory Method aplicado en createCup() y createLid().

Oracle. (2025). Java SE 17 Documentation.
<https://docs.oracle.com/en/java/javase/17/>. Referencia para la API de ArrayList, instanceof y manejo de excepciones.

Beck, K. (2002). Test Driven Development: By Example. Addison-Wesley. Referencia para la escritura de pruebas unitarias con JUnit.

Claude (Anthropic). (2026). <https://claude.ai>

La referencia más útil fue Effective Java de Joshua Bloch, específicamente los capítulos sobre herencia y composición, ya que orientó las decisiones de diseño para convertir Cup y Lid en clases abstractas y definir correctamente los contratos entre las clases.